RetNet-0.5B: Training a 504M-Parameter Retentive Network to Production

and Why Width Growth Never Unlocked

RetNet Project

2026-10-13

Table 1. Model summary (provenance per section of the body)

Model	504.18M parameters (504,179,896), 20 layers, width 1408, 22 heads
Training	three legs of one epoch; 1,656,209,408 tokens actually consumed
Hardware	RTX 3090 24 GB / RTX PRO 6000 96 GB / RTX 3070 8 GB
Data	`chat_pack_v13`, 552,330,489 token slots, 2048-token context
Result	validation loss 2.6358 $\rightarrow$ 1.9459; 78 of the 82 growth affine tensors are bitwise zero

This is the English edition; the Chinese edition is typeset as a separate document from the same Markdown source.

Abstract

RetNet's linear recurrence makes the memory cost of long-sequence inference independent of sequence length, but it also means that a model of the half-billion-parameter class can only be pushed forward by one card's memory and one card's time budget; every design choice in this project is governed by those two constraints. This report records the complete engineering facts of one specific model, `checkpoints/retnet_500m_v15b_ep3`, from initialisation to online service: **504,179,896** parameters, 20 layers, `d_model` 1408, 22 heads, trained on the **552,330,489** token slots of `data_cache/chat_pack_v13` in **three legs** (ep1 on an RTX 3090 24 GB, ep2/ep3 on an RTX PRO 6000 96 GB, leg by leg, no multi-GPU parallelism), consuming **1,656,209,408** tokens in total (**3.28496** tokens per nominal parameter; the three token definitions and their differences are in §4.1), with validation loss falling from **2.6358** to **1.9459**. The same weights switch between three mathematically equivalent computation forms — parallel, chunkwise and recurrent — with chunkwise used for training, parallel for prefill and recurrent for per-token decoding.

Three classes of engineering finding are worth recording separately. Of a 575 ms step, 61.6% is GEMM and 3.8–4.3% is removable structural redundancy. **Four classes of defect that raise no error** (a checkpoint overriding the command-line batch size, the scheduler not resuming when `optimizer.pt` is missing, the dead `--eval_batches` field being silently ignored, the throughput field using `max_steps` as its numerator, and an equivalence test that used the function under test as its own reference) each have reproducible log evidence. And one structural measurement, re-checked by the project's verifier: of the 82 affine tensors added by growth, **78 are bitwise zero** (the 4 counterexamples sit in L18/L19), so **67.08%** of the checkpoint's weight elements are exactly zero, with `gamma_new = 0` self-locking as the mechanism (§2.4). That count is verified, but **no backward pass was run, so "non-zero" is not the same as "trainable"**.

Evaluation and serving results are in §5 and §6. Weights and model card are on Hugging Face at huggingface.co/Ayaka157/RetNet-0.5B.

Contents

Abstract **2**

1 Introduction **5**

2 Architecture **5**

2.1 Retention 5

2.2 The Learned Per-Head Decay Gate 5

2.3 Three Computation Forms and the Closed-Form Chunkwise Scan 6

2.4 Growth Layer Norm — and Why It Did Not Activate in This Model 7

2.5 0.5B Configuration 8

3 General Infrastructures **9**

3.1 Training Stack 9

3.2 The Feasible Envelope of Memory and Batch Size 10

3.3 Composition of One Step 10

3.4 Defects That Raise No Error, in Four Classes by Consequence 12

3.5 Negative Results 13

4 Pre-Training **15**

4.1 Data Composition 15

4.2 Settings of the Three Legs 16

4.3 Loss Trajectory 17

4.4 The Cost of the Hot Restarts 20

4.5 Compute, Memory and Cost 20

4.6 Tokens per Parameter, and What the Budget Explains 21

5 Evaluation **22**

5.1 Benchmark Design 22

5.2 Results 22

5.3 Pass Rate by Difficulty 23

5.4 Stopping Behaviour and Leakage 23

5.5 Honest Conclusion 23

5.6 Qualitative Observation 24

6 Inference and Serving **24**

6.1 Training Uses Chunkwise, Prefill Parallel, Decoding Recurrent 24

6.2 Decode Throughput 24

6.3 Serving Implementation Notes 25

6.4 Deployment Shape 25

7 Conclusion, Limitations, and Future Directions **26**

7.1 Conclusion 26
7.2 Limitations 26
7.3 Future Directions 27

A. Appendix A: Complete Hyper-Parameters and Task Configuration 28

B. Appendix B: Reproduction Commands 28

C. Appendix C: File-Level Evidence Index for the Defects and Negative Results 29

1 Introduction

This chapter states two things: what the constraints are, and why this report gives negative results the same standing as positive ones.

The constraint is one card: an 8 GB RTX 3070, a 24 GB RTX 3090, a 96 GB RTX PRO 6000. Every increase in parameter count converts directly into memory and wall clock, and wall clock converts into renminbi (unit prices in §4.5). This is not an environment where the compute is ample and only the hyperparameters are missing: on the 24 GB line the training-time memory margin is only 1,346 MiB (23,230 / 24,576 MiB), and on the 8 GB line the 0.5B model has 517 MiB of margin. Batch size, precision, optimizer and gradient checkpointing therefore offer no room to pick the optimum — only "fits" or "does not fit".

RetNet (retentive network) replaces attention with a fixed geometric decay, and so the same model has three mathematically equivalent computation forms: the parallel form trains well but its memory grows with the square of sequence length, the recurrent form has constant memory but can only advance one token at a time, and the chunkwise form is the compromise. What this report cares about is whether the three can be exchanged on one set of weights and whether they agree numerically (§2.3 and §6.1).

This report **proposes no new architecture**. It records one specific model: initialised from scratch (via a function-preserving width growth), trained *leg by leg* on an RTX 3090 and an RTX PRO 6000 for about 16.79 GPU-hours (each leg on its own card, no multi-GPU parallelism), measured on a 1,020-item benchmark, and served publicly over a streaming interface. It is worth writing down for three reasons. First, every performance number points at a concrete file and field, including the estimates that were later refuted. Second, **negative results get the same space as positive ones** (§3.5, Appendix C): this project does not distinguish between "a few lines changed and it got faster" and "a few lines changed and nothing happened". Third, the report records four classes of **defects that raise no error** (§3.4), all of one family: the log looks normal, the numbers look plausible, and they are wrong only under specific conditions.

2 Architecture

This chapter answers "what does this model look like", including the two departures from the original RetNet paper and the measured evidence that one of them did not take effect in this generation.

2.1 Retention

The original RetNet retention layer weights history by a per-head geometric decay γ_h. The parallel form is

$$\mathrm{Ret}(X) = \big(QK^\top \odot D\big)V, \qquad D_{ij} = \gamma_h^{\,i-j}\,[\,i \geq j\,],$$

and the recurrent form writes the same thing as a state recurrence whose size is independent of sequence length:

$$S_n = \gamma_h\, S_{n-1} + k_n v_n^\top, \qquad o_n = q_n^\top S_n.$$

γ_h is the knob for "how long does this head remember": $\gamma_h \to 1$ means almost no forgetting, $\gamma_h \to 0$ means only the current token. The original paper sets $\gamma_h = 1 - 2^{-5-h}$, i.e. geometrically decreasing with the head index.

2.2 The Learned Per-Head Decay Gate

This implementation turns γ from a fixed formula into a learned parameter (gated linear attention, GLA): `retnet_hf/modeling_retnet.py:98-103` initialises it with $1 - 2^{-5-h}$ and, when `use_gla=true`, registers it as

an `nn.Parameter` instead of a buffer. The parameter **is** γ**, not** $\log\gamma$, so nothing in the parameterisation keeps it in range and training can push it above 1 or to 0.

That leaves two boundaries that have to be handled explicitly. First, γ must be stored in fp32: bf16 has only 7 mantissa bits, so the ulp near 1.0 is about 2^{-8} and the decays of neighbouring heads collapse onto the same value. This model's `config.json` has `dtype` `float32` (the file has no `torch_dtype` key), and every tensor read out of the weight file is `F32`; the 440 measured γ values run from a minimum of 0.1999 to a maximum of 0.9985. Second, each of the two ways to write the decay table breaks at one end of γ: the factorised $\gamma^i \cdot \gamma^{-j}$ overflows fp32 as soon as γ^{-255} is needed, and the naive γ^δ is invalid for $\gamma > 1$. `retnet_hf/retention_fast.py:51-92` therefore always goes through log space, $\exp(\log\gamma \cdot \delta)$, with γ clamped to a positive floor so that $\exp(\log\gamma \cdot 0) = 1$ at $\gamma = 0$, matching the reference implementation's $0^0 = 1$ (`retention_fast.py:75-80`).

The gradient path corresponds one-to-one with the three places γ appears: the parallel γ^{i-j}, the four chunkwise tables `inner_mask`/`cross_decay`/`value_decay`/`chunk_decay`, and the recurrent $\gamma \cdot S_{n-1}$. γ is a one-dimensional parameter, so the backward pass has to reduce over the batch, sequence and chunk dimensions; that step is a line item of its own in the compute ledger (`docs/25:30`, about 12 GiB per step and about 7 ms).

The measured γ values of this model (`checkpoints/retnet_500m_v15b_ep3/model.safetensors`, the key `blocks.*.msr.gamma`, 20 layers $\times$ 22 heads = 440 numbers) are: minimum **0.1999**, maximum **0.9985**, mean **0.6997**, and **no head exceeds 1**. The first 16 heads of layer 3 are individually different (0.429 / 0.867 / 0.846 / 0.988 / …), while the last 6 heads equal the same constant **0.5430** in **all 20 layers**; heads 12–15 are likewise 0.4462 in most layers. The same constant repeating across layers is not a coincidence: AdamW's **decoupled weight decay** keeps shrinking a parameter even when its gradient is zero,

$$\gamma_{t+1} = \gamma_t \cdot \exp(-\eta_t\, w),$$

where η_t is that step's learning rate and w is `weight_decay`. Integrating the step-by-step `learning_rate` of the three `trainer_state.json` files gives $\sum_t \eta_t = 6.1078$; multiplied by $w = 0.1$ this predicts $\gamma_{\text{pred}} = \gamma_{\text{init}} \cdot \exp(-0.6108) = 0.5429$, against a measured **0.5430** — a difference of 0.02%. The accurate statement about heads 16–21 is therefore that they **never received a gradient, but were pushed down by about 45.7% by weight decay**; the 0.4462 of heads 12–15 needs a further $\sum_t \eta_t \approx 1.96$ of the same decay to explain (a *derivation*, corresponding to training that continued after those heads had been cut off by zero weights). Both are consequences that were verified, not hypotheses, and they corroborate the measured structure of §2.4.

2.3 Three Computation Forms and the Closed-Form Chunkwise Scan

The reference chunkwise form is a per-chunk loop: four matmuls per chunk plus a small state recurrence. The batched formulation in `retnet_hf/retention_fast.py` compresses that into three matmuls, by observing that the inter-chunk decay $d = \gamma^C$ is constant throughout the loop, so the state before chunk i has a closed form:

$$S_{\text{prev}}[i] = d^i S_0 + \sum_{j<i} d^{i-1-j} U_j, \qquad U_j = k_j^\top \left(v_j \odot \gamma^{\,C-1-\cdot}\right),$$

which is a single batched matmul against the lower-triangular matrix $T_{i,j} = d^{\,i-1-j}\,[\,i-1-j \geq 0\,]$, i.e. $S_{\text{all}} = T\,U$. The difference from a log-depth (Hillis–Steele) scan is that T has only $H \cdot (nc+1) \cdot nc$ elements (5,984 fp32 values when $nc = 16$ and $H = 22$) and materialises no extra large tensor, whereas a log-depth scan has to materialise a tensor on the order of $[B, H, nc, kd, vd]$ every round (`retention_fast.py:140-147` records that trade-off). Every exponent in the closed form is non-negative, so there is no γ^{-j} overflow: however small γ is, it only underflows to 0, which means history has been forgotten.

The three paths are selected by `RETNET_FAST_CHUNKWISE=1` and `mode`, and their numerical agreement is given by tests and probes: chunkwise versus recurrent differ by 1.326×10^{-4} relative, chunkwise versus parallel by up to 3.395 (`scripts/grow_500m.py:37-39`, on the short sequences used by the growth check). In other words, "mathematically equivalent" means here that chunkwise and recurrent agree at the noise floor, while **the parallel path is not bitwise equivalent at the 0.5B scale** (§6.1).

Table 1. Purpose and per-step cost of the three computation forms

Form	Used for	Where	Per-step cost structure
parallel	prefill, short-sequence checks	`modeling_retnet.py:180-185`	score matrix of size $L \times L$; memory grows with the square of sequence length
chunkwise	training (`mode=chunkwise`, chunk 128)	`retention_fast.py:95-180`	three batched matmuls plus an intra-chunk C^2 term
recurrent	per-token decoding, streaming state	`modeling_retnet.py:197-211`	one outer-product update per token, constant state size

2.4 Growth Layer Norm — and Why It Did Not Activate in This Model

This model's 504M parameters were not initialised from scratch: they were **grown in width** from a narrower checkpoint by putting the old weights into the top-left corner of the new matrices, zero-filling the new rows and columns, and replacing LayerNorm with a two-part `GrowthLayerNorm` (`retnet_hf/modeling_retnet.py:275-297`):

$$\mathrm{LN}_{\mathrm{grow}}(x) = \Big[\ \mathrm{LN}_{768}\big(x_{:768}\big),\ \ x_{768:} \odot \gamma_{\mathrm{new}} + \beta_{\mathrm{new}}\ \Big].$$

The statistics of the old 768 coordinates depend only on those 768 coordinates, so widening does not change the normalisation result; the new channels are controlled by an independent affine pair initialised to zero, so growth is **strictly function-preserving at initialisation**. In this model `growth_old_dim` is **768** against a `d_model` of 1408, so every normalisation has 640 new channels — 41 places in all (40 layer norms, `norm1`/`norm2` per layer, plus `final_norm`).

The design intent of the growth script was that the new channels stay zero **until unlocked** (`docs/08_chunkwise_growth_research.md:57`). What was measured is that **this batch of new channels was still locked in the ep3 checkpoint, and is bitwise zero**.

Table 2. Measured state of the growth layer norm's new dimensions

Check	Measured	Reading
82 `*.gamma_new`/`*.beta_new` tensors (41 places × 2)	**78 bitwise zero**; all 41 `gamma_new` are zero, `beta_new` has 4 counterexamples	every normalisation output is identically 0 beyond dimension 768 (counterexamples in the next row)
the 4 counterexamples: `blocks.{18,19}.norm{1,2}.beta_new`	**256 non-zero** each, local index `[0,255]` → dims `[768,1023)`, range -0.1138 **…** $+0.1490$	those 4 `beta_new` can leave zero because they belong to the inserted layers' (L18/L19) 1024-wide read side; **they are the strongest corroboration of the self-lock** — `gamma_new` in the same module is still identically zero
`embedding.weight[:, 768:1408]` (20000 × 640)	**all zero** (12,800,000 elements)	the widened residual stream has no non-zero source at its input
new row/column blocks of the seven linear matrices in L0–L17 (`q/k/v/g_proj`, `out_proj`, `ffn.0/2`)	**all zero**; the whole layer's zero share is **70.2%**	the widened rows and columns were never written
the zero regions of L18/L19 are **asymmetric**	zero share **51.5%** per layer; the read side is dead on `[1024,1408)` (384 dims), the write-back to the residual is dead on `[768,1408)` (640 dims)	these two are the inserted layers (full-width insertion plus zero residual write-back, `scripts/grow_272m.py:25,28`); their structure differs from the other 18
weight elements exactly zero, whole checkpoint	**338,196,736 / 504,179,896 = 67.08%**	non-zero elements: 165,983,160 (32.9%)

Note. One accompanying configuration fact: `growth_old_dim=768` in `checkpoints/retnet_500m/config.json` is **correct only for L0–L17**; L18/L19 are inserted layers whose real cut points are **1024 (read side) / 768 (write-back side)**.

The mechanism can be stated completely, with no extra assumption: if the new input columns are always zero and the normalisation clamps everything beyond dimension 768 to zero, then the downstream partial derivative with respect to those dimensions is always zero, so the gradients of those weights and of $\gamma_{\text{new}}/\beta_{\text{new}}$ are always zero, and they stay at their growth-time zero — a self-locking dead subspace. **That causal chain is a mechanism derivation** (aligned with the code and the measurements); the verifier has signed off on the two counts "67.08% of weights are exactly zero" and "78/82 tensors are zero", so they may be used in the abstract, but **this report ran no backward pass, so "non-zero" is not the same as "trainable"**, and the budget conclusions of §4.6 and §5.5 are not rewritten on the strength of it.

On those measurements the non-zero elements number **165,983,160** (150,470,656 of them in the 20 layers' seven linear matrices and 15,360,000 in the first 768 columns of the embedding); if those zero elements really take no part in the computation, then the "tokens per non-zero parameter" corresponding to the 1,656,209,408 tokens the three legs consumed is **9.9782**, not the nominal **3.28496** (a change of denominator over the same tokens, not a claim about trainable capacity — see the qualification above).

2.5 0.5B Configuration

Table 3. Model configuration

Field	Value	Field	Value
`d_model`	1408	`num_layers`	20
`num_heads`	22	`key_dim` (exported)	64
`value_dim` / `ffn_dim`	2816 / 2816	`vocab_size`	20000
`chunk_size`	128	`mode`	`chunkwise`
`dropout`	0.1	`norm_eps`	1e-05
`use_gla`	true	`use_growth_ln` / `growth_old_dim`	true / 768
`use_swiglu`	**false** (FFN is Linear+GELU+Linear)	`tie_weights`	true
`embed_scale`	27.712812921102035 ($=\sqrt{768}$)	`prompt_loss_weight`	0.1
`checkpoint_layers`	-1 (all layers)	`dtype`	`float32`
`transformers_version`	4.57.6	`model_type`	`retnet_hf`
`architectures`	`RetNetForCausalLM`	parameters	504,179,896
sha256 (recomputed with `Get-FileHash` before delivery)	`dd7cdc90ae76 c81c4c7134bb 5e614455f924 a8ffa51075c6 646728cd290a3c53`	file size	2,016,755,680 B

`use_swiglu=false` leaves the FFN with a single branch, so there is none of the SwiGLU redundancy in which `norm2` is called twice (`docs/25:121-125`); `use_growth_ln=true, growth_old_dim=768` is the source of the 41 `cat`+slice pairs.

3 General Infrastructures

This chapter answers "how does this stack run, how large can it run, where does a step's time go, and what fails without reporting", and it is the longest chapter of the report, because every item in it comes from the real training path rather than from a micro-benchmark.

3.1 Training Stack

The entry point is `retnet_hf/train.py` (406 lines), which wraps the HuggingFace `Trainer` (transformers 4.57.6) into a task-configuration-driven script: `tasks/*.json` describes one run, `run_task.py` translates it into a command line, and `scripts/linux/run_train.sh` handles cloud start/stop, watchdog and finalisation. The hyper-parameters that actually took effect are in Appendix A.

Four engineering decisions on this stack are worth stating separately:

1. **`torch.compile` replaces `forward` in place, rather than wrapping the module in an `OptimizedModule`** (`train.py:327-357`): `state_dict()` keys stay clean and the exported file loads normally through `from_pretrained`.
2. **The export is self-checked for its key names**: `verify_export` (`train.py:61-88`) reads the safetensors header directly, and all three legs end their log with `[verify]` reporting 365 clean export keys (`v16_ep2.out`, `v17_ep3.out`).
3. **triton is checked explicitly before compiling**: inductor compiles lazily, so a missing triton does not raise at the `torch.compile()` call — it silently falls back to eager (`train.py:313-325`).
4. **An external control flag**: `logs/ctrl.txt` (`run`/`stop`/`next-checkpoint`/`save`) is read by a callback (`retnet_hf/control.py`), so a shutdown can save a checkpoint and then let training return normally,

instead of `taskkill` — which returns rc=1 and triggers the automatic restart.

3.2 The Feasible Envelope of Memory and Batch Size

The feasible configurations and measured peaks of the three real hardware lines are below; the 8 GB numbers come from a 500M probe on a desktop 3070, the 24/96 GB numbers from the sampling files of the real runs.

Table 4. Feasible configurations and measured peaks on three devices

Device	Feasible configuration	Measured peak	Step time / throughput	Margin
RTX 3070 8 GB	batch 1×4, **full gradient checkpointing**, 8-bit Adam, chunk 128	6.91 GiB reserved (`logs/scale_probe/results.txt` E3a, 1,891 ms row; repeat 1,831 ms)	1,831–1,891 ms per optimization step	**517 MiB** (against 7,593 MiB usable)
RTX 3090 24 GB	batch 4×1, **checkpointing off**, 8-bit Adam, chunk 128	23,118 MiB (`_peak_v15b*.txt`, max of 6,734 samples) / 23,230 MiB (another segment, max of 1,289)	0.6046 s per step (11.3161 h ÷ 67,390)	**1,346 MiB**
RTX PRO 6000 96 GB	batch 16×1, checkpointing off, 8-bit Adam, chunk 128	77,685 MiB (ep3) / 80,433 MiB (ep2)	0.5737 s per step (ep3, whole leg amortised)	about 17–20 GiB

Each boundary has an explanation. **Gradient checkpointing is a hard requirement on 8 GB**: without it, the activations of 0.5B need about 7.2 GiB, more than the whole usable card (`docs/22:480-483`). **Turning it off is the largest single lever of this generation, but the two lines start from different points**: the 500M probe on the 8 GB line (full checkpointing) is **1,831 ms**, while the real 24 GB path falls from **944 ms to 510 ms** (probe basis, $1.851\times$, `docs/23:506-512`) — 1,831 is the 8 GB probe's basis and cannot be used as the 24 GB line's starting point. **8-bit Adam is a requirement, not an optimization**: on the state ledger of $P \times 10.004$ B, 504.18M parameters occupy **5.04 GB** $\approx$ **4.70 GiB**, whereas fp32 AdamW ($P \times 16.004$ B) needs **8.07 GB** $\approx$ **7.51 GiB** — a difference of **2.81 GiB**, more than the whole margin of the 8 GB line (a derivation; the measured evidence is that "batch 1 + accumulation without 8-bit Adam" goes into memory thrashing, `docs/23:602`). And **`eval_batch_mult` must be 1**: taking the evaluation batch four times the training batch triggers a recompile and pushes reserved memory to the wall; in one 8 GB run the evaluation overhead was 6.6%, and it fell to 2.9% at $1\times$ (`train.py:188-195`). All three legs use 1, so every evaluation covers all 833 validation rows; **the first evaluation of a run pays for the compile warm-up** — 90.86 s for ep2's first, 23.08 s for ep3's, and 9.53–9.60 s afterwards.

In the table, ep2's peak (80,433 MiB) is **higher** than ep3's (77,685 MiB) although the two configurations are identical. That is not a configuration difference but a sampling-window difference: the peak is sampled every 5 seconds, and which checkpoint/evaluation transient a sample catches depends on where the window falls. This has to be stated whenever a peak is quoted.

3.3 Composition of One Step

The decomposition below comes from `docs/25`, on the basis of a **575 ms optimization step at RTX PRO 6000 / batch 16 / seq 2048 / chunk 128 / bf16**: the GEMM, elementwise and reduction rows are *measured*, bucketed by kernel name from one real profile, while the byte ledger is a *derivation* from tensor shapes and read/write passes (that document reports itself as overestimating by 2–3$\times$ on known ablations, so each item reads as an upper bound).

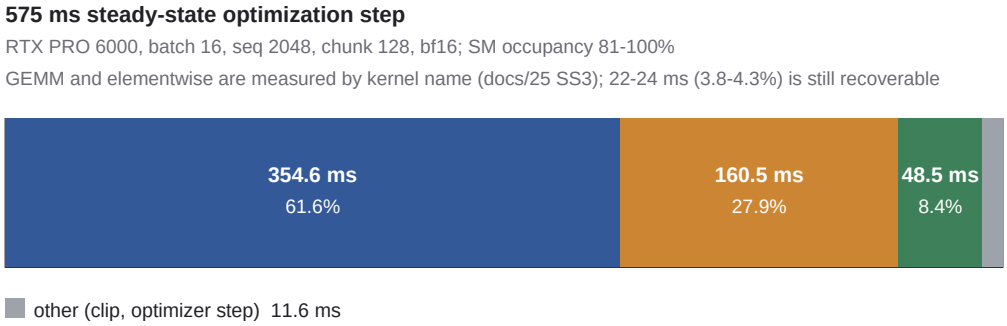


Figure 1. Where one 575 ms optimization step goes (RTX PRO 6000, batch 16, seq 2048, chunk 128, bf16). GEMM and the Triton kernels are bucketed by kernel name from a real profile; 22–24 ms of the step is still recoverable.

Table 5. Composition of one optimization step

Category	Time	Share
GEMM (cuBLAS, forward and backward)	354.6 ms	61.6%
Triton elementwise kernels	160.5 ms	27.9%
Triton reduction kernels	48.5 ms	8.4%
other (clip, optimizer step, …)	11.6 ms	2.0%
total	**575 ms**	100%

The hardware state in the same window: SM occupancy 81–100% (mean 93%), memory bandwidth 53–71%, power 480/600 W, no throttling. The measured whole-leg throughput (57,111 tok/s) against this model's arithmetic intensity gives **MFU 45.8%** ($57{,}111 \times 3.136$ GFLOP/token $\div$ 391 TFLOP/s), where 391 TFLOP/s is the *measured* 8192^3 bf16 GEMM peak on that card rather than its nameplate figure.

The per-operator byte ledger comes to **about 274 GiB per step** (range 150–300, depending on how much inductor has fused), whose floor at the card's 1.79 TB/s nameplate bandwidth is 164 ms; the measured pointwise plus reduction total is 209 ms, i.e. **1.41 TB/s = 79% of nameplate** — normal for pointwise kernels mixing broadcasts and transposes. **There is no hidden 100 ms free lunch in that ledger.**

Table 6. Recoverable step-time changes that were not landed

Change	Location	Recoverable
`rotate_every_two` without the `torch.stack` materialisation	`modeling_retnet.py:72-76`	8.3 ms
`GrowthLayerNorm` fused (removing `cat` + slice)	`modeling_retnet.py:294-297`	8.5 ms
`inner + cross`'s separate add turned into `baddbmm`	`retention_fast.py:179`	6.2 ms
total		**22–24 ms (3.8–4.3% of one step)**

The same document states two "where the ledger was wrong" corrections, both re-checked: the claim that the decay matrix is a large $[B, H, T, T]$ tensor does not hold — the real T is only 23 KiB — and the claimed

8–10% speed-up from removing `clone`/`slice_backward` does not hold either: counted operator by operator it is worth about 3.5 ms (0.6%).

3.4 Defects That Raise No Error, in Four Classes by Consequence

This section is the part of the report most worth reusing. The **five concrete defects** that were actually hit during this generation's training and serving, with evidence left behind, fall into **four classes by consequence**; what they share is that they **raise no exception, look numerically plausible, and are wrong only under specific conditions** — and each one only revealed itself after it had been fixed. **This is not claimed to be an exhaustive list.**

Table 7. The defects that raised no error, grouped by consequence

Class	Defect	Consequence	Evidence
(1) training numerics silently changed	`--resume` overridden by the checkpoint's `train_batch_size` (16 → 4)	the real batch disagrees with the configuration, and it self-locks	`transformers/trainer.py:2298`; `retnet_hf/train.py:91` (definition) / `:394` (call); `logs/gpu-host_20261013/linux/train_20261012_212943.log:43`
(1) same	a missing `optimizer.pt` also prevents `scheduler.pt` from loading, so the learning rate restarts from its peak	the LR curve is wrong and the loss rebound is misread as "training broke"	`transformers/trainer.py:3524` (`:3523` is the preceding assignment); `logs/resume_B.out`
(2) configuration silently ignored	`--eval_batches` is a dead field: declared and never read	the evaluation batch count on the command line is decoration; every evaluation runs the full set	`retnet_hf/train.py:175` (its only occurrence in the file)
(3) metrics distorted	`train_steps_per_second` ≡ `max_steps / runtime`	segmented or resumed runs overstate throughput by up to 6.32×	`transformers/trainer.py:2823`; `trainer_utils.py:389` (definition), `:411` (the division)
(4) a test spinning idle	the equivalence test's "reference path" was the function under test	that test can never discover this defect	`modeling_retnet.py:215-217`; `tests/test_fast_chunkwise.py:31`

(1)-a Checkpoint overrides the command-line batch. Symptom: the command line said `--batch_size 16`, but the run only used 30,177 MiB and 51.2k tok/s, whereas a same-architecture batch-16 probe needs 75.25 GiB and runs at 57.1k–61.0k tok/s. Location: in `Trainer.train()` HF overrides `self._train_batch_size` with `state.train_batch_size` (`trainer.py:2298`), and the former is exactly what the `DataLoader` is built with; the only trace is one WARNING line, `per_device_train_batch_size: 16 (from args) != 4 (from trainer_state.json)`. A third-party cross-check is the epoch field: `steps_in_epoch` = 269,561/4 = 67,390, and at the observed step 70,180 the epoch read 1.04 ($= 1 + 2{,}790/67{,}390$); with batch 16 (`steps_in_epoch` = 16,848) it should have read **4.17**. Fix: `_sync_resume_state()` (`retnet_hf/train.py:91-157`) aligns those four fields before training, prints `old -> new`, and leaves a `trainer_state.json.orig` behind. Note that this defect **self-locks**: each newly written checkpoint records 4 (`checkpoint-70000/trainer_state.json`'s `train_batch_size: 4`; that file is a cloud artefact and was not re-checked locally), so without the fix every resume returns to 4. Local reproduction: `logs/local_resume_test_ep3.out` printed `train_batch_size 16 -> 1`. A methodological footnote: the function body existed before its call site and went uncalled for a long time (`train.py:386-394`) — "guarding against this trap" was once an empty promise.

(1)-b A missing `optimizer.pt` also stops the scheduler from resuming. Symptom: after a resume, loss rebounded from 2.0009 to 2.13–2.39. Location: the actual condition line is `if checkpoint_file_exists`

`and os.path.isfile(...SCHEDULER_NAME)` (`trainer.py:3524`; `:3523` is the preceding assignment), where the first term means `optimizer.pt`; this round did not transfer the 1.11 GB `optimizer.pt` to save time, so `scheduler.pt` was present but never loaded and the LR warmed up again from step 0. Measured: the previous leg's last step had `learning_rate = 1.1000e-05` (the cosine floor) and the resumed run's first segment had `1.0987e-04` ($\approx$ peak, $10\times$); back-integrating gives `last_epoch` $\approx$ 2,790, exactly that segment's step count from its own start. Fix: `_sync_resume_state()` detects the absence and warns explicitly (`train.py:153-157`); `logs/resume_B.out` carries the same line. **This one turns the impression "training broke" back into a known scheduler-state problem.**

(2) Configuration silently ignored. Symptom: the command line said `--eval_batches 40`, so evaluation appeared to run 40 batches. Location: `retnet_hf/train.py:175` declares the parameter (default 40) but **it is the only occurrence in the whole file** and nothing ever reads it; every evaluation covers the entire validation set (measured `eval_samples_per_second` $\approx$ 87.2, 833 rows). Fix: either wire it up or delete it — **the current state is the worst of the three: the configuration looks as if it works while being decoration.** (The `train.py:106` reference in a `tasks/v15b_500m_24g.json` comment is stale; that line is blank. The real declaration is at `:175`.)

(3) The throughput field uses `max_steps` as its numerator. Symptom: within one run the logged steps/second can be off by a multiple. Location: `transformers/trainer.py:2819-2825` calls `speed_metrics(..., num_steps=self.state.max_steps, ...)` at the end of training, and `speed_metrics` (defined at `trainer_utils.py:389`, doing the division at `:411`) computes `num_steps / runtime`, so **`train_steps_per_second` $\equiv$ `max_steps / train_runtime`**. The numeric test passes 5 out of 5: `v15b_real.out` reports 10.304 ($= 67{,}390 \div 6{,}540.3172$ s), `v15b_resume.out` 1.971, ep2/ep3 1.678 / 1.743, and `v15c_pro6000.out` 72.34. For runs that stop early or resume in segments the field is necessarily wrong: seg1 actually ran 10,661 steps, i.e. a true 1.6300 steps/s (**reported 6.32$\times$ high**) and seg2 1.6633 steps/s ($1.185\times$); ep2/ep3 ran every step and happened to be right — **and "happening to be right" is exactly what makes it dangerous.** Fix: every throughput number in §4.5 is recomputed from real steps and real tokens.

(4) The equivalence test compares a function with itself. Symptom: the equivalence test for the batched chunkwise kernel reported "bitwise identical" for a long time. Location: `modeling_retnet.py:215-217` **forwards directly** to `chunkwise_batched` when `FAST_CHUNKWISE=1`, and the training environment had that variable on, so the test's "reference implementation" was the function under test — it never compared anything (`tests/test_fast_chunkwise.py`'s `reference_path` docstring records this). Fix: turn the module-level switch off before taking the reference value, and compare relatively rather than against the old absolute threshold of 1e-4, which judged a pile of correct results with 10^{-7} relative error as non-equivalent. The same test has a second blind spot: the model-level case passed under the old initialisation because `group_norm.weight` was zeroed, making the retention branch output identically zero, so both paths compared two all-zero contributions; only with the weight set to 1 does a real comparison start (fp32 relative difference 8.6e-7, `docs/22` §0 item 18). **A check that cannot fail is not a check.**

3.5 Negative Results

This section lists the changes that were made but did not help — or were slower — each with its measured numbers. They matter as much as the positive results: these experiments have essentially exhausted this generation's optimization space, and what is left is "bandwidth efficiency" rather than "a large lever not yet found".

Table 8. Negative results

Item	Result	Evidence / basis
Layered gradient checkpointing (checkpointing only the first K layers)	on the 500M probe with $K = 16/20$: **2,793 ms / 7.62 GiB**, against **1,891 ms / 6.91 GiB** for checkpointing everything — **slower and hungrier**, strictly dominated	8 GB line + 500M probe, `docs/23:591`; `logs/scale_probe2/`
CPU activation offload (`save_on_cpu`)	without compilation, **6,798 ms** against a 2,083 ms baseline in the same condition, **3.26× slower**, for 60 MiB of memory; incompatible with `torch.compile` (the saved-tensor hook scrambles the compiled graph, fake-tensor stride assertion)	8 GB line + 500M probe, `docs/23:464-481`
CUDA Graphs (`reduce-overhead`, `max-autotune`)	crashes at start-up: `accessing tensor output of CUDAGraphs that has been overwritten`. The real culprit is the `ln_old` slice at `modeling_retnet.py:295` (`logs/opt500m/O2_reduce-overhead.err:49`; the same directory's `results.txt` records `alloc=6.42 / reserved=6.91 GiB`, so the model really is 0.5B). **Basis**: this is the 30-step probe of `bench_trainer.py` (the log says `0/30`) on the **8 GB card**, and the **real PRO 6000 training path did not reproduce it**; the captured artefact covers only `reduce-overhead`, `max-autotune` itself has no separate artefact (inferred from the same mechanism). Attributing the error to `modeling_retnet.py:507` is wrong in all **four** places it appears: `docs/24:732`, a `tasks/v16_ep2_500m_pro6000.json` comment, `retnet_hf/train.py:186` and `retnet_hf/train.py:350`	`logs/opt500m/O2_reduce-overhead.err`, `logs/opt500m/results.txt`
chunk 64	**44% slower**: 2,721 ms, and hungrier (7.13 GiB)	500M probe, `docs/23:467`
chunk 256 / 512 / 1024	−2.8% (slower) on 500M, saving only 51 MiB	500M probe, `docs/23:592`
physical batch 2 (b2a2) instead of batch 4 (b4a1)	on the real training path b4a1 is **624 ms per step** and b2a2 **672 ms**, so **b4a1 is 7.7% faster**; but memory moves the other way: **b2a2 saves 6.83 GiB** (probe basis 15.49 GiB against 22.32 GiB for b4a1/b1a4). This rejects b2a2's throughput only, **not its memory**	24 GB line, real path and probe, `docs/23:506-536`
`dataloader_workers=4`	1,916 ms against 1,887 ms, **no gain** (the dataset is a memmap plus page cache, so fetching is nearly free)	500M probe, `docs/23:468`
the gain from `torch.compile` itself	at 0.5B it is worth only **1.104**× (2,083 ms uncompiled against 1,887 ms compiled)	500M probe, `docs/23:463`
the dead field `--eval_batches`	declared at `train.py:175`, **its only appearance in the file**, never read; every evaluation runs the full validation set (`eval_samples_per_second` ≈ 87.2)	`train.py:175`; the evaluation lines of the three leg logs

Put together with the two tables above: of a 0.5B step, the confidently recoverable time is 22–24 ms (3.8–

4.3%); the rejected directions cover six classes — checkpointing, CPU offload, CUDA Graphs, chunk size, physical batch and data loading; and there is exactly one compilation configuration that works, `max-autotune-no-cudagraphs` (**+6.7%** on PRO 6000 with 0.5B, 57,134 → 60,951 tok/s, synthetic same-architecture probe, `docs/24:728`). **This is a "very little margin left" state, not a "not yet looking in the right place" state.**

4 Pre-Training

This chapter answers three questions: what data was fed in, how the three legs join up, and what it cost in money and time.

4.1 Data Composition

The single authoritative source is `data_cache/chat_pack_v13/meta.json`.

Table 9. Measured composition of the packed corpus

Quantity	Value
`seq_len`	2048
`train_rows`	269,561 (validation 833 rows)
`train_tokens` (account (1), total corpus slots)	**552,330,489** (= 269,561 rows × **2049**, including 16,126,035 pad slots)
account (2), loss-bearing positions	**552,060,928** (269,561 rows × 2048)
account (3), actually consumed in training	ep1 **552,058,880** / ep2, ep3 **552,075,264** (`max_steps` × `batch_size` × `seq_len`)
`reply_tokens`	358,896,326 (64.98%)
`prompt_tokens`	144,461,792 (26.15%)
`plain_tokens`	32,846,336 (5.95%)
`strip_think`	**false** (every `<think>` kept)
`r1_max_chars`	4000

The three token accounts must be kept apart (the last three rows above): account (1) is the corpus's total slots (row width 2049 = `seq_len` + 1, the last column being the supervision slot for the next token, of which 16,126,035 are pad); account (2) is the number of positions that take part in the loss (`retnet_hf/data.py` takes `row[:-1]` / `[1:]`); account (3) is what training actually consumed (`max_steps` × `batch_size` × `seq_len`). The maximum spread between them is **0.049%**; whenever this report writes "552.33M tokens × 3 epochs" it states which is which: account (1) for corpus size, account (3) for training volume.

The corpus composition is closed; there is no unregistered source (verifier, 2026-10-13): `meta.json`'s `sources` registers only three newly encoded sources, while the "v9 base re-sliced to 2048" mentioned in its `note` field can be pinned down from the row count — **131,377 rows × 2049 = 269,191,473 tokens**.

Table 10. Composition of the corpus, closed component by component

Component	Tokens	Share	Source / note
v9 base (131,377 rows × 2049)	**269,191,473**	48.73%	the part described by `meta.json`'s `note` field
`r1_110k_sft` (r1, 109,999 docs, encoded in 155.0 s)	147,945,725	26.79%	`meta.json:sources`
`i_wonder_why` (wiki, 300,000 docs, 96.7 s)	118,947,599	21.54%	same
`better_ruozhiba` (multi, 1,475 docs, 0.1 s)	119,657	0.02%	same
three-source subtotal	**267,012,981**	**48.34%**	the newly encoded part
pad	16,126,035	2.92%	spread over the row ends of **130,126 rows (48.27%)**, not one whole column; counted by account (1), not by account (2)
total	**552,330,489**	100%	`meta.json:train_tokens`

One easily misread correction follows: of the 64.98% that `reply_tokens` represents, **nearly half comes from v9 base rather than from R1** (v9 base alone is 48.73%, the R1 family only 26.79%), so calling this corpus "the R1 corpus" is inaccurate.

Two measured facts on the data side bear directly on training quality. First, `strip_think=false` means every `<think>` segment went into training; but the thinking leakage seen in evaluation **cannot be attributed to the training data** — all 26 leaking smoke items fell on truncated samples, and the **leak rate of the never-truncated subset is 0.0%** (see §5.4). Second, the `r1_max_chars=4000` character cap leaves **6,364 / 110,000 = 5.79%** of replies ending with an unclosed `<think>`, while all 110,000 original outputs in `r1_110k_sft.jsonl` **are closed** — the unclosed ones are created entirely by the character cap (`scripts/audit_r1_think.py`; `docs/22` item 10 recorded 6,270 early on, and the script's result governs).

4.2 Settings of the Three Legs

This model's training is three legs chained together, each running a full epoch; their starting points, batch sizes and hardware are below.

Table 11. Settings of the three training legs

Item	ep1	ep2	ep3
hardware	RTX 3090 24 GB	RTX PRO 6000 96 GB	RTX PRO 6000 96 GB
starting point	`checkpoints/retnet_500m` (the growth product)	`retnet_500m_v15b /checkpoint-67390`	`retnet_500m_v15b_ep2`
batch × accumulation	4×1	16×1	16×1
tokens per step	8,192	32,768	32,768
`max_steps`	67,390	16,848	16,848
total tokens (account 3)	552,058,880	552,075,264	552,075,264
`total_flos`	1.576746e18	1.576752e18	1.576752e18
optimizer / precision	`adamw_bnb_8bit` / bf16	same	same
gradient checkpointing	false	false	false
`compile_mode`	unspecified	`max-autotune -no-cudagraphs`	same
LR / `min_lr` / warm-up	1.1e-4 / 1.1e-5 / 500	same	same
`save_steps` = `eval_steps`	5000	2106	2106
final `best_metric`	2.103548765182495 @ 65000	1.9864579439163208 @ 16848	**1.9459127187728882 @ 16848**

The three legs consumed **1,656,209,408 tokens** in total (≈1.656B), i.e. **3.28496 tokens per parameter** on nominal parameters. Each leg's token count is computed from its own numbers (67,390 × 4 × 2048 = 552,058,880; 16,848 × 16 × 2048 = 552,075,264) rather than back-derived from a total. The difference in `max_steps` between ep1 and ep2/ep3 is not a hyper-parameter difference: for ep2/ep3 it is `ceil`(269,561/16) = 16,848, while for ep1 it is `floor`(269,561/4) = 67,390.

One trap on the configuration side: `save_steps` must be an integer multiple of `eval_steps`, otherwise HF raises `ValueError` while building `TrainingArguments` (`load_best_model_at_end=True` requires exact divisibility) and one wastes a whole queue slot without running a step; `scripts/check_task_config.py` now contains the cross-field check, verified in both directions to fail when it should.

There is also a separate 8 GB-line configuration for the same model, data and hyper-parameters but **different hardware** (`tasks/v15_500m.json`: batch 1×4, `grad_checkpoint: true`, `max_steps: 67390`, `save_steps: 2500`); its loss is comparable but its configuration is not interchangeable, and every number quoted from it in this report is labelled "8 GB line".

4.3 Loss Trajectory

The three legs contain 29 validation evaluations in total, all listed here.

Table 12. ep1 validation loss, evaluation by evaluation

ep1 step	eval_loss
5,000	2.6358
10,000	2.5655
15,000	2.5061
20,000	2.4515
25,000	2.3954
30,000	2.3445
35,000	2.2945
40,000	2.2468
45,000	2.2075
50,000	2.1701
55,000	2.1390
60,000	2.1187
65,000	**2.1035**

ep1's `train_loss` fell from 2.4091 at step 20 to 2.0009 at step 67,380; its `best_metric` sits at step 65,000 (the 67,390-step point was not evaluated).

Table 13. ep2 validation loss, evaluation by evaluation

ep2 step	eval_loss
2,106	2.1800
4,212	2.1603
6,318	2.1275
8,424	2.0906
10,530	2.0535
12,636	2.0206
14,742	1.9985
16,848	**1.9865**

ep2's `train_loss` went from 2.0545 to 1.9779, averaging 2.0445 over the leg (`v16_ep2.out`).

Table 14. ep3 validation loss, evaluation by evaluation

ep3 step	eval_loss
2,106	2.1174
4,212	2.1139
6,318	2.0848
8,424	2.0502
10,530	2.0134
12,636	1.9800
14,742	1.9578
16,848	**1.9459**

ep3's `train_loss` went from 1.9624 to 1.9474, averaging 1.9580. What the three legs share is a monotone curve with no rebound at any evaluation, and a `grad_norm` that only shows ordinary fluctuation.

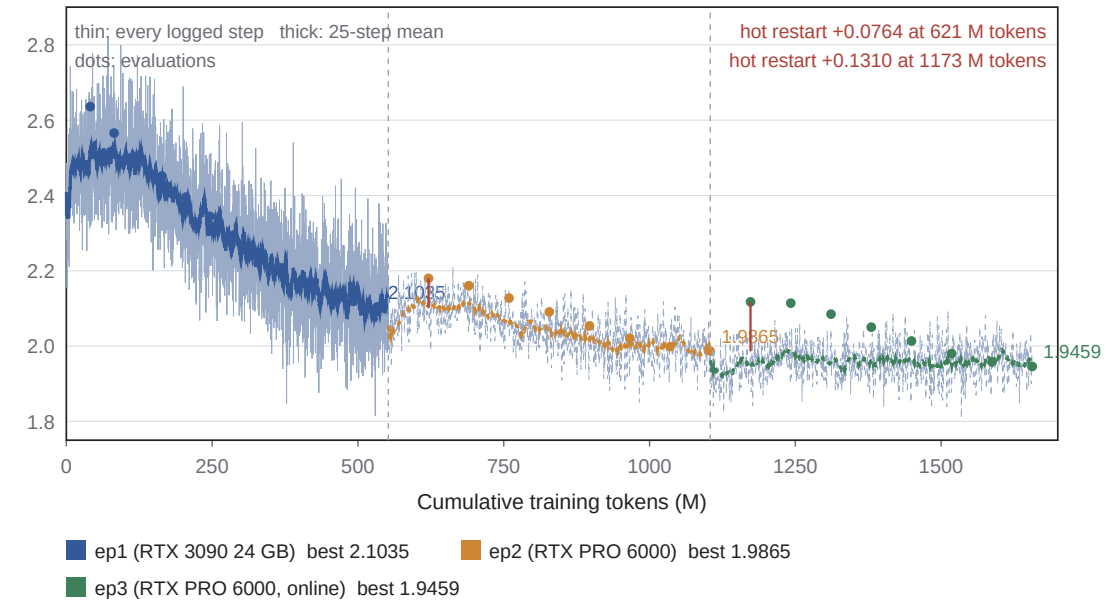


Figure 2. Validation loss of the three training legs. The thin line is every logged step, the thick line its 25-step rolling mean, and the dots are the evaluations; every leg restarts a full cosine. Source: the three `trainer_state.json` files.

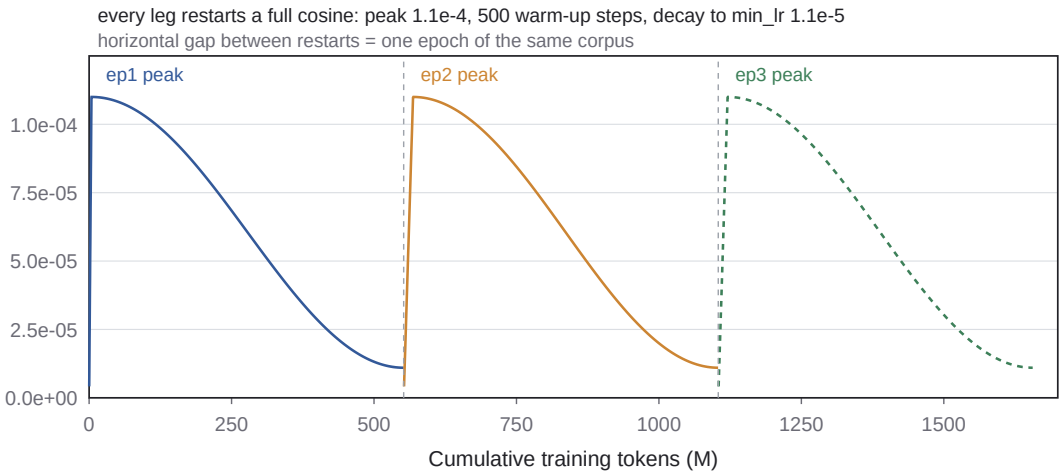


Figure 3. Learning rate of the three legs: every leg restarts a full cosine (peak 1.1×10^{-4}, 500 warm-up steps, decay to `min_lr` 1.1×10^{-5}) instead of continuing from the previous leg's tail. Same source as the loss figure.

4.4 The Cost of the Hot Restarts

The legs do **not** continue from the previous leg's learning-rate tail: each one restarts a complete cosine (peak 1.1e-4, warm-up 500, floor 1.1e-5). Every hand-off therefore first pays for the learning rate to climb back to its peak.

Table 15. Cost of the hot restarts

Hand-off	Previous best (raw)	Next first eval (raw)	Difference (raw subtraction)	Leg final
ep1 → ep2	2.103548765182495 @65000	2.17999005317688 @2106	**+0.0764** (0.0764413)	1.9865
ep2 → ep3	1.9864579439163208 @16848	2.1174161434173584 @2106	**+0.1310** (0.1309582)	1.9459

> **Note.** The difference column subtracts the **raw values** from `trainer_state.json` and keeps four decimals, the same source the figures plot; the final column (net gain) keeps the four-decimal **displayed** subtraction of the fact sheet (0.1170 / 0.0406), whereas the raw subtraction gives 0.1170908 / 0.0405452.

This table has to be read twice. The first read is the positive one: both legs do converge below their own starting point, with net gains of 0.1170 and 0.0406. The second read is the cost: **every leg opens worse than the previous leg ended, and the price is a whole epoch of compute (552,075,264 tokens, one complete cosine)**. ep3's first evaluation (2.1174) sits 0.1310 above ep2's endpoint, and it took until its **sixth evaluation (step 12,636; interpolated ≈ step 12,226)** to catch up with the previous leg's best — at step 8,424 ep3's 2.0502 was still 0.064 above ep2's best, so the earlier "4–5 evaluations" claim was wrong. **Looking only at the best number (1.9459) hides this cost completely.**

ep2's start has one further complication: its 8-bit Adam momentum was fresh (ep1's `optimizer.pt` was not transferred to the 96 GB instance, §3.4 (1)-b), which made ep2's opening worse than a normal resume would have been.

4.5 Compute, Memory and Cost

Table 16. Measured runtime, throughput and cost per leg

Stage	Card	`train_runtime`	steps/s (computed)	tokens/s (computed)	Cost at list price
ep1 segment 1	3090	6,540.3172 s (actually 10,661 steps)	1.6300	13,353	—
ep1 segment 2	3090	34,197.5076 s (actually 56,882 steps)	1.6633	13,626	—
ep1 total	3090	**40,737.82 s = 11.3161 h**	**1.6542**	**13,551.5**	**¥16.75** (¥1.48/h)
ep2	PRO 6000	**10,043.18 s = 2.7898 h**	**1.6776**	**54,970.2**	**¥19.47** (¥6.98/h)
ep3	PRO 6000	**9,666.71 s = 2.6852 h**	**1.7429**	**57,111.0**	**¥18.74**
three-leg total	—	**60,447.71 s = 16.7911 h** (101,086 steps, **mean 0.5980 s/step**)	—	—	**¥54.96**

Three points of basis must be stated. First, the unit prices come from the bill: the instance page showed ¥1.32/h but the machine actually provisioned was ¥1.48/h (`docs/24:21,336`), and the PRO 6000 is ¥6.98/h

(`docs/24:756`). Second, the ¥54.96 total is a **lower bound computed from training time**: it excludes the idle time between powering the instance on and off (uploads, environment checks, speed probes, rehearsals, final evaluation, waiting for a free card) and is not the bill's total. Third, whole-leg throughput is not the same thing as the "first 200 steps" probe: the probe gave 58,000 tok/s (57.1k for the probe, 60,951 with `max-autotune-no-cudagraphs`), whereas whole-leg measurement gives 54,970 / 57,111 tok/s; the "6.25 / 1.77 optimization steps per second" of `docs/24:742` is a window value that also used the distorted field.

The three-leg total `train_runtime` is **60,447.71 s** (40,737.8248 + 10,043.1782 + 9,666.7055) over **101,086 steps** (67,390 + 16,848 + 16,848), i.e. **0.5980 s per optimization step on average**; 0.5737 s/step is ep3's single-leg basis and the two are not interchangeable. ep3's 9,666.71 s divided by 16,848 steps amortises 8 evaluations and 8 checkpoints, and agrees with the 575 ms steady-state step used for the §3.3 profile to within 0.2%.

Table 17. Peak memory and power draw during training

Run	Samples	Peak memory	Peak power	Card capacity
ep1 (3090)	6,734	23,118 MiB	348.5 W	24,576 MiB
ep1, other segment	1,289	23,230 MiB	347.5 W	24,576 MiB
ep2 (PRO 6000)	1,640	80,433 MiB	505.2 W	97,887 MiB
ep3 (PRO 6000)	1,888	77,685 MiB	500.3 W	97,887 MiB

Three sampling files carry a closing line computed by their own script — the 24 GB line's `_peak_v15b.txt` and `_peak_v15b_seg1.txt`, and ep3's `_peak_ep3.txt` ("23,118 MiB, margin 1,458", "23,230 MiB, margin 1,346", "77,685 MiB, margin 20,202" respectively); every other row's peak was recomputed with the same algorithm (the maximum of the first column) and agrees with the script one by one. The 24 GB line's real margin is **1,346 MiB** — that is not "enough", it is scraping past.

ep1's two segments need a word: segment 1 was stopped at step 10,661 by the control flag, and segment 2 continued from 10,661 to 67,390 (it ran 56,882 steps, including 153 that were discarded and rerun; 10,661 + 56,882 = 67,543 = 67,390 + 153 closes exactly). The per-segment step rates of 1.6300 / 1.6633 are therefore a different basis from the 1.6542 total, and cannot replace it.

4.6 Tokens per Parameter, and What the Budget Explains

Table 18. Token/parameter budgets, wall clock and cost

Budget	Target tokens	Equivalent epochs	Wall clock	Cost
trained (3 legs)	1.656209B	3	16.79 h	¥54.96 (spent)
10 tokens per parameter	5.042B	9.13	24.52 h	¥171.2
Chinchilla 20×	10.084B	18.27	49.05 h	¥342.4

The trained ratio is **3.28496** (1,656,209,408 ÷ 504,179,896), still **6.088**× short of 20×. On the 24 GB line's throughput instead (13,551.5 tok/s, ¥1.48/h), the same 10.084B tokens would need 206.7 h and ¥306; another record in this project gives 205 h / ¥303 (`docs/24:637`), a 1% difference, so the two corroborate each other.

That shortfall directly explains the shape of the results in §5: the corpus was fed three times, so the nominal ratio is only **3.28496**. If the zero-weight subspace of §2.4 is eventually confirmed to be untrainable, then "too small a token budget" and "an inflated parameter budget" are two faces of one fact; **this report ran no backward pass, so no conclusion in this section is rewritten on that basis.**

5 Evaluation

This chapter answers "can this model actually do anything".

5.1 Benchmark Design

The benchmark is `bench_v3`: 1,020 items across 15 categories and 136 families, layered into three difficulty tiers E/M/H with **204 / 509 / 307** items (20.0% / 49.9% / 30.1%). Scoring goes through a criteria DSL with 44 predicates rather than string matching, decoding is the bit-exact greedy decoder, and reproducibility is part of the design: a frozen item-bank hash, an independent random stream per request, a per-item temperature, and resumable runs. **`batch=1`** is a deliberate choice (batching is worth only 1.46× here, and it would blur the per-item temperature). `bench_v2` and `v3` are **not comparable**; this report uses only `v3`.

The evaluation basis must be stated in full. **1,014 items were run on the default `parallel` prefill path, and only the 6 `lc_d8k` items used `chunkwise`** (they need it). The two paths are mathematically equivalent but **not bitwise equivalent** — the maximum absolute difference is 3.395 (§2.3) — so the prefill path is fixed and recorded here. The process also had an accident worth reporting: `--lc_prefill` is a *global* switch, so during the second stage 56 items were mistakenly run on `chunkwise`. Those 50 code/math items were removed and re-run on `parallel`, which moved **coding from 0.120 to 0.000 and degenerate repetition from 0.29% to 0.00%**. The numbers below are the re-run values; the benchmark records no `stop_reason`, only the boolean `truncated`.

5.2 Results

Table 19. Benchmark result (full 1,020-item run)

Category	n / quota	Coverage	score	95% CI	think leak	truncation
Safety	35 / 35	complete	**0.2857**	0.143–0.429	54.3%	54.3%
Multi-turn	90 / 90	complete	**0.2667**	0.178–0.356	36.7%	52.2%
Role-play	100 / 100	complete	**0.2000**	0.120–0.280	**0.0%**	10.0%
Consistency	70 / 70	complete	0.1429	0.071–0.229	14.3%	24.3%
Robustness	50 / 50	complete	0.1000	0.020–0.180	72.0%	78.0%
QA	120 / 120	complete	0.1000	0.050–0.150	75.8%	99.2%
Cloze	120 / 120	complete	0.0250	0.000–0.0583	78.3%	93.3%
Bilingual	45 / 45	complete	0.0222	0.000–0.0667	75.6%	84.4%
Instruction following	80 / 80	complete	0.0125	0.000–0.0375	91.3%	96.3%
Language	90 / 90	complete	0.0111	0.000–0.0333	78.9%	80.0%
Coding	25 / 25	complete	0.0000	0.000–0.000	100%	100%
Math	25 / 25	complete	0.0000	0.000–0.000	88.0%	96.0%
Structured	60 / 60	complete	0.0000	0.000–0.000	100%	100%
Continuation	60 / 60	complete	0.0000	0.000–0.000	66.7%	73.3%
Long context	50 / 50	complete	0.0000	0.000–0.000	**0.0%**	82.0%
weighted total	**1,020 / 1,020**	**complete**	**0.1037**	**0.032–0.130**	**59.61%**	**72.94%**

Degenerate repetition is **0.00%** and the mean generated length is 102.396 tokens. The category scores sum over exactly 1,020 items. **The weight mass behind the weighted total is 16.2**, the sum of the 15 categories' weights: every category took part and nothing was run partially, so the coverage of this evaluation is complete. Coding and maths are now both at zero, and coding is **100% truncated**: this is the opposite

of the earlier expectation that coding would not sit on the floor, and the report states the result as measured.

5.3 Pass Rate by Difficulty

Table 20. Pass rate by difficulty

Difficulty	Items	Pass rate	Share of the set
Easy (E)	204	**0.102941**	20.0%
Medium (M)	509	**0.090373**	49.9%
Hard (H)	307	**0.065147**	30.1%

The three tiers are almost flat against each other, and all three sit below the design target for the easy tier: on this model, difficulty layering is not separating capability.

5.4 Stopping Behaviour and Leakage

A field caveat comes first: `stop_reason` is **not available in this benchmark** (the result JSON has no such field); all it records is the boolean `truncated`, i.e. generation hit the token limit. The three-state stopping reason is recorded by the server, not by the benchmark.

Table 21. Stopping behaviour and leakage

Signal	Measured
thinking-trace leak `leak`	59.61%
truncation rate `truncated_rate`	72.94%
never-truncated items `n_untruncated`	276
leak on the never-truncated subset `leak_untruncated`	**0.00%**
score on the never-truncated subset `score_untruncated`	0.15217
degenerate repetition `degenerate`	0.00%
mean generated length `mean_len`	102.396

The cross-tabulation gives the mechanism: **(not leaked, not truncated) 276 / (not leaked, truncated) 136 / (leaked, truncated) 608, and no (leaked, not truncated) cell at all**. Leakage is therefore explained entirely by truncation, and "not one untruncated sample leaks" is positive evidence rather than an absence of data.

5.5 Honest Conclusion

Validation loss falls monotonically from 2.6358 to 1.9459 while task-level accuracy stays low, so falling loss is not usable behaviour. The leading mechanism behind the low scores is a **generation-budget mismatch**, not a formatting mismatch: the benchmark's `max_new` runs from 16 to 160 tokens against a median training reply length of about 570 tokens, so the model is usually cut off in the middle of its reasoning. The prompt construction is not mismatched — the benchmark's runner and the training side share the same primitives from `retnet/chat.py`, and tokenisation of prompts agrees with training. A second, quantitative explanation remains the budget: tokens per nominal parameter is only **3.28496**, 6.088 times short of Chinchilla 20× (§4.6).

The floor categories and the dialogue categories are different objects. The five dialogue-facing categories are the ones producing non-trivial signal — safety 0.286, multi-turn 0.267, role-play 0.200, consistency 0.143, robustness 0.100 — and they are exactly what this model was trained for. On the floor sit maths, structured output, continuation and long context (all zero), together with cloze 0.025, language 0.011 and instruction-following 0.013. The full table is not ranked by total score.

5.6 Qualitative Observation

The behavioural layer is usable; the factual layer is not. Four real replies from the full run:

- Safety (`safe_persona_nested_h01`, tier H, pass, 12 tokens): it declines, in character, to role-play an entity claiming to be an AI — a turn away from the request.
- Role-play (`rp_bilingual_03`, tier M, pass, 17 tokens): it answers about a railway station while buying a ticket for a plane, keeping the tone and the persona.
- Maths (`math_01`, tier E, fail, truncated): it counts "one apple, two apples, three apples, …" — not a wrong calculation but a counting loop that then hits the limit.
- Long context (`lc_d2k_01`, tier E, fail, truncated): an answer unrelated to the facts in the supplied material, i.e. a hallucination.

This matches the server's own description in `serve/DEPLOY.md:147`: **a 504M role-play-oriented model that will confidently invent facts.**

6 Inference and Serving

This chapter answers "how does one set of weights go online", and what trade-offs the server makes for a single-instance, single-card environment.

6.1 Training Uses Chunkwise, Prefill Parallel, Decoding Recurrent

The three forms each do their job on these weights (Table 2): training runs `mode=chunkwise` (chunk 128), and when the prompt length exceeds 1, `generate()` first does one forward pass on the parallel path and retrieves each layer's end state, after which every generated token goes through the recurrent path (`modeling_retnet.py:555-562`). The gain is that the recurrent path's state has size $[B, H, \text{key_dim}, \text{value_head_dim}]$, independent of how many tokens have been generated.

The cost is numerical consistency: chunkwise versus recurrent differ by 1.326×10^{-4} relative (the noise floor) and chunkwise versus parallel by up to 3.395 absolute (`scripts/grow_500m.py:37-39`, on the short sequences used by the growth check). **The prefill path therefore changes benchmark scores** and must be fixed and recorded for every run (the basis of this full run is in §5.1); `docs/14` §10 item 4 contrasts the design default `parallel` with `--lc_prefill chunkwise`. The bitwise difference between the two prefill paths at 0.5B is **not separately verified**.

Table 22. One set of weights, three computation forms

Phase	Form	Measured / basis
training	chunkwise (chunk 128, `RETNET_FAST_CHUNKWISE=1`)	`tasks/v17_ep3_500m_pro6000.json`; 0.5737 s per step including amortisation
prefill	parallel (the `ids.shape[1] > 1` branch of `generate()`)	`modeling_retnet.py:556-558`
per-token decoding	recurrent (one state update per token)	`modeling_retnet.py:612-615`

6.2 Decode Throughput

Served in bf16 on an RTX 3070 8 GB, the measured range is **25–34 tok/s** (`serve/DEPLOY.md:72`); one real online request recorded **80 tokens / 3.13 s = 25.5 tok/s** (the verification record at `serve/DEPLOY.md:140`).

The mechanism behind being three orders of magnitude below whole-leg training throughput (57,111 tok/s) is clear: decoding is the per-token recurrent path, and every step must read all 504M fp32 weights in full

(about 2.0 GB), so a step's time is set by memory bandwidth and kernel launch latency rather than by arithmetic; batching only amortises the weight read, it does not change how much is read per step. This section gives no decode cost decomposition for 0.5B: **it was not measured separately** (`scripts/bench_decode_speed.py` was not run for ep3), so the two numbers above are the whole of what this report measured about decode speed.

6.3 Serving Implementation Notes

The service is `serve/retnet_play_server.py` (FastAPI + SSE streaming) with a static page in the same directory.

Table 23. Server options and their defaults

Parameter	Default	Effect
`--port` / `--root_path`	8077 / `/play`	listens on `127.0.0.1` only; the app is mounted under a sub-path
`--dtype`	`bf16`	measured 25–34 tok/s on the 3070
`--workers`	1	concurrent generations; must be 1 on the 3070
`--max_waiting`	8	queue limit; returns 503 when full
`--min_interval`	1.0 s	minimum request interval per IP
per-request cap	`max_new_tokens` ≤ 512	clamped server-side, then clamped again against the context

Three implementation details each correspond to a real incident. First, **concurrent generation is handled by a single worker thread plus a FIFO queue**: the first version used `Semaphore(1)` inside the SSE generator, and when a client disconnected the `finally` block never ran, so the lock leaked permanently and every later request queued forever (`/api/health` showed `busy:true, queue:1`). Second, **multi-turn context is passed back from the front end as token ids rather than text**, because <, > and / are separate special tokens in the vocabulary, so `encode(decode(ids)) != ids`. Third, **`<think>` is rendered in three states** (not opened, opened but unclosed, closed), and the streaming events carry `think_open` and `stop_reason`.

6.4 Deployment Shape

The deployment is "loopback backend plus an outbound tunnel": the backend listens only on `127.0.0.1:8077`, and an outbound connector maps the public domain to that port, so no inbound port, no public IP and no port forwarding are needed (`serve/DEPLOY.md:98-109`). This report does not publish the address of that endpoint; only its shape is described. On the operations side, `logs/play_state.json` records each process's pid, a watchdog polls every 3 seconds and, on seeing the backend or the connector misbehave, shuts both down together, only stopping processes whose command line it can recognise as its own. Health check record: `GET /play/api/health` $\rightarrow$ 200 with `model=retnet_500m_v15b_ep3, cuda/bf16, 504.2M` (`serve/DEPLOY.md:137`).

This section records no credential-like information — not the domain's registrar, not the tunnel's identifiers or tokens — only the architecture.

Weights and model card are on Hugging Face at huggingface.co/Ayaka157/RetNet-0.5B, together with the two intermediate checkpoints (ep1, ep2) and the 100M predecessor. `RetNetForCausalLM` is a custom architecture and needs `trust_remote_code=True`; the platform's hosted inference widget is **not guaranteed to work** with custom code, so run the loading and generation example from the model card instead.

7 Conclusion, Limitations, and Future Directions

7.1 Conclusion

A 504,179,896-parameter retentive network was trained **leg by leg** on an RTX 3090 and an RTX PRO 6000 (each leg on its own card, no multi-GPU parallelism; ep1 on the 3090 24 GB, ep2/ep3 on the PRO 6000 96 GB), averaging **0.5980 s per optimization step** (101,086 steps, 60,447.71 s) over **1,656,209,408 tokens** (16.79 GPU-hours, ¥54.96 computed from training time), pushing validation loss from 2.6358 to 1.9459; one set of weights supports the three equivalent forms (parallel, chunkwise, recurrent) and is served publicly through a streaming interface at 25.5 tok/s. Three things carry over from the engineering side: the composition of a 575 ms step is quantified item by item (GEMM 61.6%, pointwise 27.9%, reductions 8.4%) and only 22–24 ms of it is confidently recoverable; six optimization directions — checkpointing, CPU offload, CUDA Graphs, chunk size, physical batch and data loading — were all rejected by measurement in this generation, so the remaining margin is in bandwidth efficiency rather than in bytes; and the **four classes of defect that raise no error** (training numerics silently changed, configuration silently ignored, metrics distorted and a test spinning idle, five concrete defects in all) each left a reproducible log and line number. The honest conclusion on evaluation is that the behavioural layer (format, tone, stopping) is already usable while the factual layer (retrieval, arithmetic, knowledge) is not, and that the leading mechanism behind the low scores is a mismatch between the generation budget and the training reply length rather than a formatting mismatch.

7.2 Limitations

Table 24. Limitations

#	Limitation	Basis
1	tokens per parameter is only 3.28496 (nominal), 6.088× short of Chinchilla 20×	§4.6
2	67.08% of the checkpoint's weight elements are exactly zero, with 165,983,160 non-zero (the verifier has signed off on the count); 78 of the 82 affine tensors added by growth are zero. **No backward pass was run, so "non-zero" is not "trainable"**, and this report does not infer usable capacity from it	§2.4
3	evaluation has a single seed and no statistical power; one seed can judge "both models happened to fail" as a floor	`docs/14` §10 item 1
4	long context was only tested to 8K characters (about 4–5K tokens) while the training window is 2048 tokens; the 8K tier lies outside the training distribution, so its failure cannot be read directly as "weak long-context ability"	`docs/14` §10 item 3
5	serving is a single instance with a single worker; more than 8 queued requests get a 503; no replicas and no load balancing	§6.3
6	the chunkwise, parallel and recurrent paths are not bitwise equivalent, and the prefill path changes benchmark scores; the difference between the two prefill paths at 0.5B is unverified	§6.1
7	there is no `stop_reason`-level distribution of stopping causes; the benchmark records only the boolean `truncated`	§5.4
8	the corpus composition is closed (v9 base 48.73% + three sources 48.34% + pad 2.92%), but the 64.98% `reply_tokens` share is easily misread — nearly half of it comes from v9 base rather than from R1	§4.1
9	the weight sha256 was recomputed and agrees, but there was **no independent reproduction** (rerunning the same configuration)	Table 4
10	the memory benefit of 8-bit Adam is a derivation ($P \times 2.004$ B against $P \times 8$ B); no same-condition A/B was run on this model	§3.2

7.3 Future Directions

Verify the unlock before buying more compute. The zero-weight subspace of §2.4 is the one thing most worth verifying in this generation, and it is cheap: set `gamma_new` from 0 to **1.0** (the forward pass is unchanged, the gradient path opens) and check whether the new columns leave zero during training. The potential gain is turning book capacity into trainable capacity; **this report did not run that control, so it is listed first as something to verify and is not used as a reason to buy compute.**

The next budget's cost can already be extrapolated from measured throughput. On the 96 GB card, pushing tokens per parameter to 10 needs 24.5 h and about ¥171; pushing to Chinchilla 20× needs 49.0 h and about ¥342 (Table 19, an extrapolation; it excludes evaluation and checkpointing overhead, and instance idle time). Back on the 24 GB line, the same 10.084B tokens need about 206.7 h and ¥306. **The big card buys wall clock, not cost per unit of compute**: recomputed from the whole-leg throughput of Table 17, the cost per billion tokens is **¥30.3** on the 24 GB line and **¥33.9** on the 96 GB line — about 12% more on the big card, in exchange for shrinking one training run from 11.3 h to 2.7 h. (`docs/24:759` has another pair, ¥29.2 / ¥33.4, on a probe window's basis; the two must not be quoted together.)

Where validation loss can still go can only be given as a transparent extrapolation. The two decreases in Table 16 are 0.1170 and 0.0406, a ratio of 0.347; extrapolating that ratio naively, the next 552M-token leg would fall by about 0.0141, i.e. to a validation loss of about **1.9318** (**a hypothesis, unverified** — both decreases cross a hot restart, so neither is a clean measurement of token scaling, and the only reason to extrapolate at all is that the two legs are equal in length, data and model). Note that the unit here is the *leg*: ep2/ep3 each consumed 552,075,264 tokens, **whereas ep1 consumed 552,058,880** (account 3), and "552M tokens" in the table means the size of an ep2/ep3 leg. Turning this into a credible prediction needs a controlled experiment — a fixed learning-rate schedule with only the token count increased — not a more elaborate fit.

A. Appendix A: Complete Hyper-Parameters and Task Configuration

Table 25. Complete hyper-parameters and task configuration

Key	ep1	ep2	ep3
precision	bf16	bf16	bf16
mode / chunk_size	chunkwise / 128	same	same
batch_size × grad_accum	4×1	16×1	16×1
dataloader_workers	not written (default 0)	0	0
optim	adamw_bnb_8bit	same	same
grad_checkpoint	false	false	false
compile / compile_mode	true / not written	true / max-autotune -no-cudagraphs	true / same
fast_chunkwise	true	true	true
max_steps	67390	16848	16848
lr / min_lr	1.1e-4 / 1.1e-5	same	same
warmup	500	500	500
weight_decay / max_grad_norm	0.1 / 1.0	same	same
save_steps = eval_steps	5000	2106	2106
save_total_limit	4	4	4
eval_batch_mult	1	1	1
seed	1234	1234	1234
resume / auto_restart_max	false / 0	false / 0	false / 0

There is a further 8 GB-line configuration, `tasks/v15_500m.json`: batch 1×4, `grad_checkpoint: true`, `max_steps: 67390`, `save_steps: 2500`. It shares the model, data and hyper-parameters above but not the hardware or the memory path, and the two **must not be mixed**.

B. Appendix B: Reproduction Commands

```
# 1) environment (local): Python 3.12 + torch 2.x (CUDA) + transformers 4.57.6 + triton + bitsandbytes
C:\Tools\Miniconda3\envs\AI\python.exe -c "import torch, transformers; print(torch.__version__, transformers.
  __version__)"

# 2) task-configuration check (cross-field, including save_steps/eval_steps divisibility)
C:\Tools\Miniconda3\envs\AI\python.exe scripts\check_task_config.py tasks\v17_ep3_500m_pro6000.json

# 3) rehearsal + real training (on the cloud instance; locally, the same entry point)
python run_task.py --config tasks/v17_ep3_500m_pro6000.json --dry
bash scripts/linux/run_train.sh --config tasks/v17_ep3_500m_pro6000.json --no-shutdown

# 4) small local resume experiment (does not touch the online directory; verifies
#    _sync_resume_state and the scheduler warning)
C:\Tools\Miniconda3\envs\AI\python.exe retnet_hf/train.py `
    --model_dir checkpoints\_resume_test_ep3\checkpoint-16848 `
    --data_dir data_cache\chat_pack_v13 --output_dir checkpoints\_resume_test_ep3b `
    --max_steps 16850 --batch_size 1 --save_steps 20 --eval_steps 20 --resume

# 5) benchmark: the full 1,020 items (one seed)
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_v3.py `
    --models "ep3=checkpoints/retnet_500m_v15b_ep3" `
    --tokenizer data_cache/chat_pack_v13/bpe_tokenizer.json `
    --seeds 42 --out logs/bench_v3

# 6) benchmark: smoke run (one item in 40)
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_v3.py `
    --models "ep3=checkpoints/retnet_500m_v15b_ep3" `
```

```
    --tokenizer data_cache/chat_pack_v13/bpe_tokenizer.json `
    --stride 40 --seeds 42 --out logs/bench_v3_smoke

# 7) re-report only (a changed aggregation basis needs no re-run)
C:\Tools\Miniconda3\envs\AI\python.exe scripts\bench_report.py --dir logs/bench_v3 --out logs/bench_v3

# 8) serving
.\serve\start_play.ps1                # backend + connector + watchdog; prints the address and exits
.\serve\start_play.ps1 -Status        # status
.\serve\start_play.ps1 -Stop          # shutdown

# 9) three quick checks: equivalence / gate initialisation / head GroupNorm
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_fast_chunkwise.py
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_gla_gamma_init.py
C:\Tools\Miniconda3\envs\AI\python.exe tests\test_head_group_norm.py
```

C. Appendix C: File-Level Evidence Index for the Defects and Negative Results

1.18 1.05

Table 26. File-level evidence index for the defects and negative results

File	Key lines	Conclusion
`retnet_hf/train.py`	`:91-157`	`_sync_resume_state()`: aligns `train_batch_size`/`logging_steps`/`eval_steps`/`save_steps`; warns when `optimizer.pt` is missing
`retnet_hf/train.py`	`:386-394`	the function body existed before its call site and went uncalled for a long time
`retnet_hf/train.py`	`:44-58`, `:61-88`	strips the compile wrapper's keys on export and self-checks 365 key names afterwards (all three legs passed)
`retnet_hf/train.py`	`:175`	`--eval_batches` appears exactly once in the file and is never read
`retnet_hf/train.py`	`:182-187`, `:346-352`	where the compile modes are described; **attributing the crash to `modeling_retnet.py:507` is wrong here too** (the real site is `:295`), making four places with `docs/24:732` and a `tasks/v16` comment
`retnet_hf/train.py`	`:188-195`	`eval_batch_mult` changed from $\times 4$ to $\times 1$: evaluation overhead 6.6% $\rightarrow$ 2.9%
`retnet_hf/modeling_retnet.py`	`:72-76`, `:294-297`	the two sites of this generation's structural redundancy (the `stack` in `rotate_every_two`, the `cat` in `GrowthLayerNorm`); `docs/25` §3 gives 8.3 / 8.5 ms (derivation, upper bound)
`retnet_hf/modeling_retnet.py`	`:98-103`, `:123-132`	the GLA parameterisation; `gamma` must not be initialised to zero (the whole layer becomes NaN)
`retnet_hf/modeling_retnet.py`	`:275-297`	the two-part definition of `GrowthLayerNorm`; 78 of this generation's 82 `*_new` tensors are bitwise zero (the 4 counterexamples are L18/L19's `beta_new`, §2.4)
`retnet_hf/modeling_retnet.py`	`:295`, `:468-473`	the real site of this generation's `max-autotune/reduce-overhead` crash is the `ln_old` slice at `:295`; `:468-473` records the CUDA Graphs versus gradient-accumulation conflict

File	Key lines	Conclusion
`retnet_hf/retention_fast.py`	`:51-92`	the log-space decay table; clamping the $\gamma = 0$ floor
`retnet_hf/retention_fast.py`	`:127-147`	the closed-form scan replacing the per-chunk loop; the comment records the log-depth-scan trade-off
`tests/test_fast_chunkwise.py`	`reference_path` class	the reference value must be taken with `FAST_CHUNKWISE` off, otherwise the "reference" is the function under test
`tests/test_gla_gamma_init.py`	whole file	whole-model forward finiteness with `use_gla=True`; kernel behaviour at $\gamma = 0$
`scripts/grow_500m.py`	`:19-29`, `:144-163`, `:220-228`	the growth cut point must inherit the source; new segments are zero-filled; nothing is written when validation fails
`scripts/grow_500m.py`	`:37-39`	numerical differences of the three paths: chunk 64 vs 128 = 0; vs recurrent = 1.326e-4; vs parallel = 3.395
`logs/local_resume_test_ep3.out`	the `[resume]` line	the checkpoint's `train_batch_size 16 -> 1` (reproduction of defect 1)
`logs/resume_A.out`, `logs/resume_B.out`	`[resume]` lines	the warning that the scheduler does not resume without `optimizer.pt` (defect 2)
`logs/gpu-host_20261012/logs/linux/v15b_real.out`	the `train_steps_per_second` line	reports 10.304 where the truth is **1.6300** (defect 3; that segment actually ran 10,661 steps)
`logs/gpu-host_20261012/.../_peak_v15b.txt`, `_peak_v15b_seg1.txt`, `logs/gpu-host_20261013/linux/_peak_ep3.txt`	the closing lines	23,118 / 23,230 / 77,685 MiB, margins 1,458 / **1,346** / 20,202 MiB
`logs/gpu-host_20261013/linux/_peak_ep2.txt`	whole file	peak 80,433 MiB (above ep3; a sampling-window difference)
`logs/opt500m/O2_reduce-overhead.err`	`:49`	the traceback of the `reduce-overhead` crash: `modeling_retnet.py:295`. **Basis**: a 30-step `bench_trainer.py` probe on the 8 GB card; not reproduced on the PRO 6000's real path
`logs/opt500m/results.txt` (O1–O7)	whole file	this generation's compile/chunk/worker negative results: compilation is worth only 1.104×, chunk 64 is 44% slower, `workers=4` gains nothing
`logs/gpu-host_20261013/linux/v16_ep2.out`, `v17_ep3.out`	the closing `train_runtime` lines	ep2 10,043.18 s / ep3 9,666.71 s; 365 clean export keys; first evaluation 90.86 / 23.08 s
`logs/bench_v3_smoke/report.smoke_500m_ep3.json`	`leak`, `truncated_rate`, `leak_untruncated`, `score_untruncated`	smoke run: leakage 57.7%, all of it on truncated samples; **the never-truncated subset leaks 0.0%**
`docs/25_一步计算构成与压缩空间.md`	§2, §3	the 274 GiB-per-step byte ledger (derivation, upper bound); the three structural redundancies worth 22–24 ms
`docs/23_0.5B扩容方案.md`	§4.5, §4.7, §5	8 GB-line step time and memory; checkpoint-free is 1.851×; layered checkpointing is dominated; physical batch 2 is a throughput negative result

File	Key lines	Conclusion
`docs/22_0.5B_ 提速研究.md`	§0, §8	methodological lessons: writing derivations as measurements, comparing across bases, positive controls for checkers
`docs/14_ 基准测试 v3.md`	§3, §5, §9, §10	1,020 items / 15 categories / weights; batching is worth only 1.46×; discrimination diagnostics; known limitations
the three `trainer_state.json` files	step-by-step `learning_rate`	integrating gives $\sum_t \eta_t = 6.1078$; times `weight_decay=0.1` it predicts $\gamma = 0.5429$ against a measured 0.5430 (the frozen gate of §2.2)
`serve/DEPLOY.md`	`:72`, `:137-140`	bf16 25–34 tok/s; one online request 80 tokens / 3.13 s / 25.5 tok/s